Software Re-Engineering

Lecture 9

Code cleaning

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

Agenda

- Code cleaning techniques

Consistent Formatting:

- Maintain a consistent code formatting style throughout your codebase.
- This includes consistent indentation, spacing, and naming conventions.

1. Vertical formatting-> Newspaper metaphor

Think of a well-written newspaper article. You read it vertically. At the top you expect a headline that will tell you what the story is about and allows you to decide whether it is something you want to read. The first paragraph gives you a synopsis of the whole story, hiding all the details while giving you the broad-brush concepts. As you continue downward, the details increase. We would like a source file to be like a newspaper article. The name should be simple but explanatory. The name, by itself, should be sufficient to tell us whether we are in the right module or not. The topmost parts of the source file should provide the high-level concepts and algorithms. Detail should increase as we move downward, until at the end we find the lowest level functions and details in the source file.



1.1 Vertical Openness Between Concepts:

Should be there are a blank lines that separate the package declaration, the import(s), and each of the functions.

Nearly all code is read left to right and top to bottom. Each line represents an expression or a clause, and each group of lines represents a complete thought. Those thoughts should be separated from each other with blank lines.



There are blank lines that separate the package declaration, the import(s), and each of the functions. This extremely simple rule has a profound effect on the visual layout of the code.

Listing 5-1

BoldWidget.java

```
package fitnesse.wikitext.widgets;

import java.util.regex.*;

public class BoldWidget extends ParentWidget {
  public static final String REGEXP = "'''.+?'''";
  private static final Pattern pattern = Pattern.compile("'''(.+?)'''",
    Pattern.MULTILINE + Pattern.DOTALL
  );

  public BoldWidget(ParentWidget parent, String text) throws Exception {
    super(parent);
    Matcher match = pattern.matcher(text);
    match.find();
    addChildWidgets(match.group(1));
  }

  public String render() throws Exception {
    StringBuffer html = new StringBuffer("<b>");
    html.append(childHtml()).append("</b>");
    return html.toString();
  }
}
```

1.2 Vertical Density:

The lines of code that are more related to each other should appear close to each other.

```java
public class ReporterConfig {

    /**
     * The class name of the reporter listener
     */
    private String m_className;

    /**
     * The properties of the reporter listener
     */
    private List<Property> m_properties = new ArrayList<Property>();

    public void addProperty(Property property) {
        m_properties.add(property);
    }
```



```java
public class ReporterConfig {
    private String m_className;
    private List<Property> m_properties = new ArrayList<Property>();

    public void addProperty(Property property) {
        m_properties.add(property);
    }
```



1.3 Vertical Distance:

•**Variable Declarations**: variables should be declared as close to their usage as possible.

```
private static void readPreferences() {
    InputStream is= null;
    try {
        is= new FileInputStream(getPreferencesFile());
        setPreferences(new Properties(getPreferences()));
        getPreferences().load(is);
    } catch (IOException e) {
        try {
            if (is ≠ null)
                is.close();
        } catch (IOException e1) {
        }
    }
}
```

For loops control variables should usually be declared within the loop statement.

```
public int countTestCases() {
    int count= 0;
    for (Test each : tests)
        count += each.countTestCases();
    return count;
}
```



Instance variables, on the other hand, should be declared at the top of the class. ecause they are used by many of the methods within the class. And because everybody should know where to go to see the declarations.

Dependent Functions: If one function calls another, they should be vertically close, and the **caller** should be above the **callee**.



Conceptual Affinity. Certain bits of code want to be near other bits. They have a certain conceptual affinity. The stronger that affinity, the less vertical distance there should be between them. As we have seen, this affinity might be based on a direct dependence, such as one function calling another, or a function using a variable. But there are other possible causes of affinity



These functions have a strong conceptual affinity because they share a common naming scheme and perform variations of the same basic task. The fact that they call each other is secondary

```
public class Assert {
  static public void assertTrue(String message, boolean condition) {
    if (!condition)
      fail(message);
  }

  static public void assertTrue(boolean condition) {
    assertTrue(null, condition);
  }

  static public void assertFalse(String message, boolean condition) {
    assertTrue(message, !condition);
  }

  static public void assertFalse(boolean condition) {
    assertFalse(null, condition);
  }
...
```

1.4 Vertical Ordering:

This means a function that is called should be below a function that does the calling.



2. Horizontal formatting

How wide should a line be? The best answer for that is "you should never have to scroll to the right".

2.1. Horizontal Openness and Density

We didn't put horizontal space for things that are more related, and we put horizontal space for things are less related (are separate).

- The writer didn’t put horizontal spaces between the function name and the opening parenthesis. This is because the function and its arguments are **closely related (more related).**
- Surround the assignment operators with horizontal spaces (left side and the right side) to clarify them. This because they **are separate (less related)**.
- Separated arguments within the function call parenthesis to clarify the comma and show that the arguments are separate.

```
private void measureLine(String line) {
    lineCount++;
    int lineSize = line.length();
    totalChars += lineSize;
    lineWidthHistogram.addLine(lineSize, lineCount);
    recordWidestLine(lineSize);
}
```

Another use for horizontal space is to clarify the **precedence** of operators.

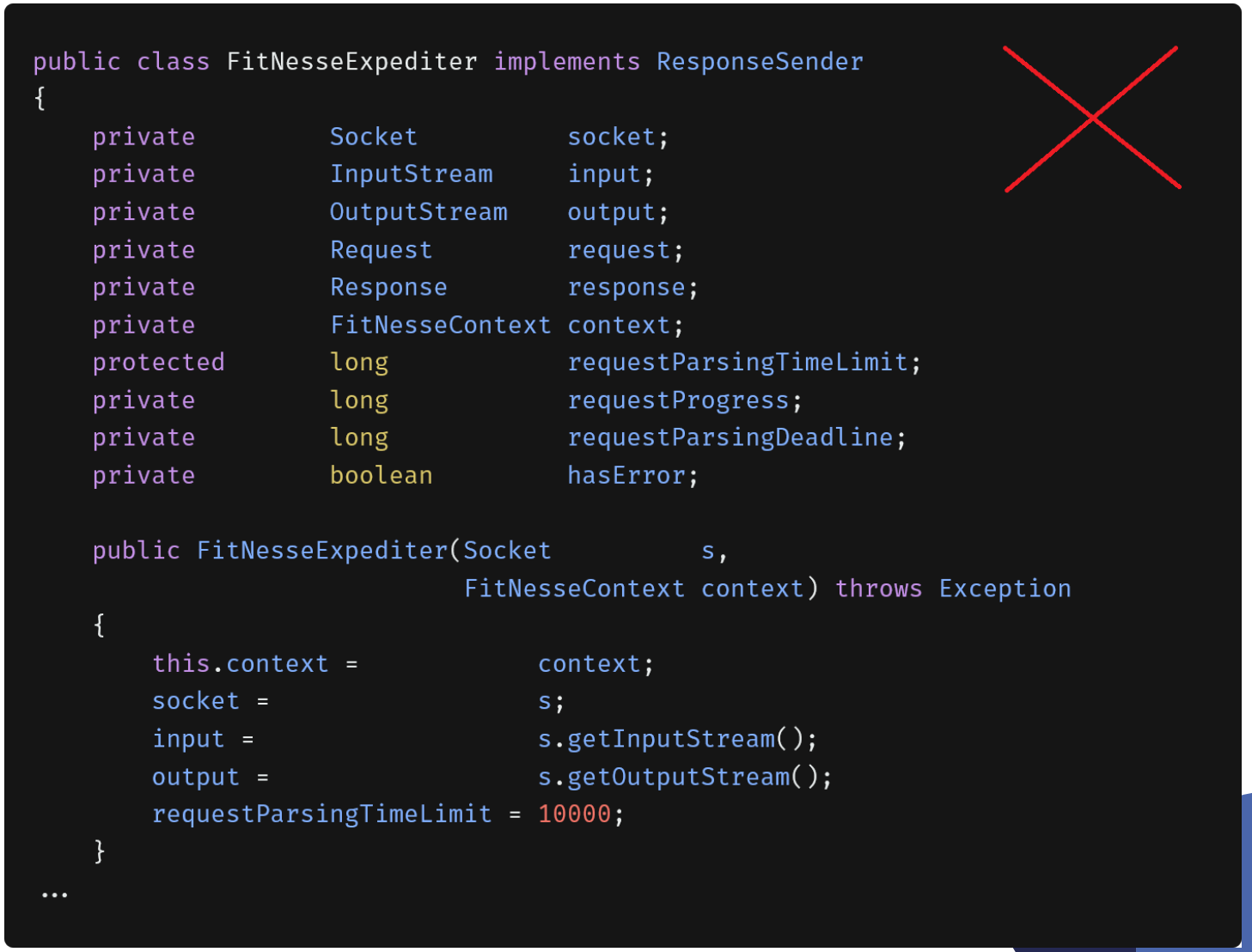
```
public class Quadratic {
    public static double root1(double a, double b, double c) {
        double determinant = determinant(a, b, c);
        return (-b + Math.sqrt(determinant)) / (2*a);
    }

    public static double root2(int a, int b, int c) {
        double determinant = determinant(a, b, c);
        return (-b - Math.sqrt(determinant)) / (2*a);
    }

    private static double determinant(double a, double b, double c) {
        return b*b - 4*a*c;
    }
}
```

2.2. Horizontal Alignment

The horizontal alignment is not useful



```
public class FitNesseExpediter implements ResponseSender
{
    private          Socket          socket;
    private          InputStream     input;
    private          OutputStream    output;
    private          Request         request;
    private          Response        response;
    private          FitNesseContext context;
    protected        long            requestParsingTimeLimit;
    private          long            requestProgress;
    private          long            requestParsingDeadline;
    private          boolean         hasError;

    public FitNesseExpediter(Socket          s,
                             FitNesseContext context) throws Exception
    {
        this.context =           context;
        socket =                 s;
        input =                  s.getInputStream();
        output =                 s.getOutputStream();
        requestParsingTimeLimit = 10000;
    }
...
```

```
public class FitNesseExpediter implements ResponseSender
{
    private Socket socket;
    private InputStream input;
    private OutputStream output;
    private Request request;
    private Response response;
    private FitNesseContext context;
    protected long requestParsingTimeLimit;
    private long requestProgress;
    private long requestParsingDeadline;
    private boolean hasError;
    public FitNesseExpediter(Socket s,
                             FitNesseContext context) throwsException
    {
        this.context = context;
        socket = s;
        input = s.getInputStream();
        output = s.getOutputStream();
        requestParsingTimeLimit = 10000;
    }
...
```



2.3. Indentation

The indentation is the left space that show the scopes of classes within the file, the methods within the classes, the blocks within the methods, and recursively to the blocks within the blocks. Each level of this hierarchy is a scope.

To make this hierarchy of scopes visible, we indent (length of left space) the lines of source code in proportion to their position in the hierarchy.

indentation rules:

- **Statements** at the level of the file, such as class declarations, are not indented (no left space)
- **Methods** within a class are indented one level to the right of the class.
- **Implementations of those methods** are indented one level to the right of the method declaration.
- **Block** implementations are implemented one level to the right of their containing block, and so on.



```
public class FitNesseServer implements SocketServer { private FitNesseContext
context; public FitNesseServer(FitNesseContext context) { this.context =
context; } public void serve(Socket s) { serve(s, 10000); } public void
serve(Socket s, long requestTimeout) { try { FitNesseExpediter sender = new
FitNesseExpediter(s, context);
sender.setRequestParsingTimeLimit(requestTimeout); sender.start(); }
catch(Exception e) { e.printStackTrace(); } } }
```



```
public class FitNesseServer implements SocketServer {
    private FitNesseContext context;

    public FitNesseServer(FitNesseContext context) {
        this.context = context;
    }

    public void serve(Socket s) {
        serve(s, 10000);
    }

    public void serve(Socket s, long requestTimeout) {
        try {
            FitNesseExpediter sender = new FitNesseExpediter(s, context);
            sender.setRequestParsingTimeLimit(requestTimeout);
            sender.start();
        }
        catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Team Rules

A team of developers should agree upon a single formatting style, and then every member of that team should use that style. We want the software to have a consistent style.



Readings

Chapter 5, **Clean Code_ A Handbook of Agile Software Craftsmanship - Robert C. Martin**